Unit 5 - Software Engineering and Development Methodologies

1 Software Failures and Lessons Learned

- Software failures can happen:
- Lack of tests
- Edgcases not considered
- Poor communication
- Lack of documentation/poor quality
    - People leave
    - Large teams
- Lack of experience/skills
- Silos
- Over time
    - Poor alignment
    - Human factor:
        - People underestimate work
        - People over estimate themselves

One well-known example is the **NASA Mars Climate Orbiter failure (1999)**.

The spacecraft was lost because two teams used different measurement systems:

- One team used **metric units (newtons)**
- Another team used **imperial units (pounds-force)**

Incorrect trajectory calculations and Orbiter crashed as entered Martian atmosphere at the wrong angle.

What went wrong?

Several issues contributed to the failure:

- Poor communication between development teams
- Lack of standardised unit validation
- Insufficient testing
- Failure to detect the error during system verification

How could better software engineering have prevented the failure?

Better practices could have prevented the problem:

- Clear documentation
- Automated checks
- More thorough testing
- Better communication

Which SDLC model was likely used?

Large aerospace projects usually use a **Waterfall development approach**.

Waterfall focuses heavily on documentation and structured development stages.
However, even structured approaches can fail if testing and validation are not performed properly.

2 Software Engineering Principles

Software engineering is the structured design, development and maintenance of software systems.

Structured development helps reduce bugs, improve security, and ensure systems meet user requirements.

Software Quality Attributes

Modern software systems are judged by several key quality attributes.

Maintainability

- Maintainability refers to how easy it is to update or modify software.
- Well-structured code, documentation and modular design make systems easier to maintain.

Scalability

- Scalability describes how well a system handles increased workload.
- A scalable system can support more users or larger amounts of data without slowing down.

Security

- Security protects software systems from cyber attacks or unauthorised access.
- Security features include authentication, encryption and vulnerability testing.

Performance

- Performance refers to how efficiently software processes data.
- High-performance software responds quickly and uses system resources efficiently.

Software Development Life Cycle (SDLC)

The **Software Development Life Cycle (SDLC)** is a structured process used to develop software systems.

Typical stages include:

1. Requirements analysis
2. System design
3. Implementation (coding)
4. Testing

5. Deployment
6. Maintenance (ongoing costs)

Using a structured lifecycle helps ensure that software is reliable and meets user needs.

Modern Software Development Trends

Software development has evolved significantly due to cloud computing and distributed systems.

Microservices

Microservices architecture breaks large systems into smaller services.

Each service performs a specific function and can be developed independently.

Advantages include:

- Easier scaling
- Faster updates
- Improved reliability

Containerisation

Tools such as **Docker** allow developers to package applications and their dependencies together.

This ensures the software runs consistently across different environments.

Container Orchestration

Platforms such as **Kubernetes** manage multiple containers by automating deployment, scaling and monitoring.

Serverless Computing

Serverless platforms allow developers to run code without managing servers directly.

The cloud provider automatically manages infrastructure and scaling.

Discussion Questions

Why do we need structured development models?

Structured development models help teams organise complex software projects.

They provide:

- Clear development stages
- Proper testing procedures
- Improved documentation
- Reduced project risk

Without structured development, software projects may become disorganised and more prone to failure.

How does software engineering balance innovation with reliability?

Software engineering balances innovation and reliability through:

- Incremental development
- Automated testing
- Continuous integration
- Frequent small updates instead of large releases

This allows companies to experiment and improve software while maintaining system stability.

---

3 Comparing SDLC Models

Three major development methodologies are commonly used in modern software engineering.

- Waterfall
- Agile
- DevOps

Each approach is suited to different types of projects.

Waterfall Model

The **Waterfall model** is a linear development process where each stage is completed before the next begins.

Characteristics

- Sequential development stages
- Clear documentation requirements
  Strict planning before development begins

Advantages

- Predictable development schedule

- Clear project scope
  Suitable for regulated industries

Disadvantages

- Difficult to change requirements later
- Problems may only be discovered late in development

Waterfall is often used in industries such as aerospace, healthcare and government software projects.

Agile Development

Agile development focuses on iterative development and continuous improvement.

Instead of delivering the entire product at once, software is released in small increments.

Characteristics

- Short development cycles (sprints)
- Continuous feedback from users
- Frequent testing and updates

Advantages

- Flexible requirements
- Faster feature delivery
- Continuous improvement

Disadvantages

- Requires strong team communication
- Less predictable timelines

Many technology companies use Agile because it allows rapid adaptation to changing requirements.

DevOps Methodology

DevOps combines **software development (Dev)** with **IT operations (Ops)**.

The goal is to automate and streamline the software deployment process.

Key DevOps practices

- Continuous Integration (CI)
- Continuous Deployment (CD)
- Automated testing
- Infrastructure automation

Advantages

- Faster software releases
- Reduced deployment errors
- Improved collaboration between teams

Disadvantages

- Requires organisational culture change
  Heavy reliance on automation tools

DevOps allows companies to release updates frequently while maintaining reliability.

Activity – Choosing an SDLC Model

Example project: **Developing a new mobile banking application**

Different development approaches may be suitable depending on the project requirements.

Waterfall example

A government-regulated banking system may use Waterfall because it requires:

- Extensive documentation
- Strict security verification
- Regulatory approval

Agile example

A start-up building a mobile banking application may use Agile because:

- Requirements may change during development
- User feedback is important
- Rapid updates are required

DevOps example

DevOps may be used to automate testing and deployment so updates can be released quickly.

---

4 Agile and DevOps in Action

Agile and DevOps help organisations improve software development speed, collaboration and reliability.

Sprint Planning Simulation

In Agile development, work is organised into **sprints**, typically lasting 2–4 weeks.

Example project: **Developing an AI chatbot**

Step 1 – User Stories

User stories describe features from the user perspective.

Examples:

- As a user, I want the chatbot to answer simple questions.
- As a user, I want the chatbot to provide customer support information.

Step 2 – Product Backlog

The backlog is a list of all tasks required to build the system.

Example backlog items:

- Chatbot conversation engine
- Natural language processing module
- User interface design
- Security testing

Step 3 – Sprint Goals

Each sprint focuses on completing a small number of backlog items.

Example sprint goal:

- Build chatbot response system
- Create basic user interface
- Test chatbot accuracy

How sprint planning differs from Waterfall

Waterfall development plans the entire system at the start of the project.

Agile development allows plans to change between sprints.

This flexibility allows teams to adapt to new requirements or feedback.

Challenges of Agile in large organisations

Large organisations may struggle to implement Agile because:

- Teams may be distributed across different locations
- Management structures may be hierarchical
- Legacy systems may limit flexibility

Adopting Agile often requires organisational culture changes.

DevOps Automation Tools

DevOps relies heavily on automation tools.

Jenkins

Jenkins is used for **continuous integration**.
It automatically builds and tests software when new code is added.

Docker

Docker packages applications into containers to ensure consistent environments.

Kubernetes

Kubernetes manages large numbers of containers and automatically distributes workloads across servers.

Continuous Deployment Example

Companies such as Netflix, Amazon and Facebook use DevOps to deploy software updates frequently.

These companies rely on:

- Automated testing
- Continuous integration pipelines

- Microservices architecture

For example, Amazon reportedly deploys software updates approximately **every 11.7 seconds**.

Frequent updates allow companies to fix bugs quickly and introduce new features rapidly.

Conclusion

Software engineering provides the structured processes needed to develop reliable and scalable software systems.

Different development methodologies serve different purposes:

- **Waterfall** structured and predictable development
- **Agile** flexible and iterative development
- **DevOps** automated deployment and continuous delivery

Modern organisations often combine these approaches depending on the project requirements.

Choosing the correct development methodology is critical for delivering high-quality, secure and maintainable software.

Ongoing costs

Sample testing

Rolling out step by step

Project of software like any other project management issue